

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



Sistemas Distribuidos

Laboratorio. S09

Que el aviso no tumbe la transferencia

Grupo: 02

Profesor: Oscar Manuel Ruiz Uscardo

Alumna: Segura Cedeño Luisa María

Semestre 2026-1

Fecha de Entrega: 24/10/2026

Que el aviso no tumbe la transferencia

1. El problema

Parte 0: Estado y saldos

Respuesta HTTP 500 Internal Server Error devuelta por el sistema al intentar realizar una transferencia mientras el servicio de notificaciones se encontraba detenido.

```
luisa@LAPTOP-SOE65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped

luisa@LAPTOP-SOE65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
-d '{"claveOrigen": "0021800000000000001", "claveDestino":"0021800000000000002", "monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 01:33:00 GMT

{"timestamp":"2026-10-18T01:33:00.289+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
```

Consulta a la API de transferencias filtrada por estado=ABONADA, mostrando que la operación financiera se quedó varada a medio camino.

```
luisa@LAPTOP-SOE65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":4,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50.00,"fecha":"2026-10-18T01:32:56.492839Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-18T01:32:59.015048Z"}]
```

Consulta de los saldos de Ana (950) y Beto (550), lo que evidencia que el dinero sí se movió físicamente entre las cuentas a pesar de que la transferencia no pudo completarse.

```
luisa@LAPTOP-SOE65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000001
{"id":6,"clabe":"0021800000000000001","titular":"Ana","saldo":950.00}
luisa@LAPTOP-SOE65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000002
{"id":7,"clabe":"0021800000000000002","titular":"Beto","saldo":550.00}
```

¿Por qué pasó?

El cliente recibe un error porque la saga dependía de una llamada HTTP síncrona para confirmar la notificación al usuario; aunque el cargo, el abono y los asientos contables ya se habían ejecutado con éxito, la transacción intentó esperar la respuesta de un servicio caído, provocando que la operación completa fallara y la transferencia quedara varada en estado ABONADA.

2. Publicar

El código clave explicado

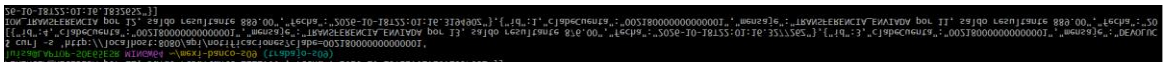
Para que la transferencia ya no dependa de notificación, cambié la saga para que publique un evento en RabbitMQ en lugar de llamar por HTTP. Estas son las partes principales:

- **eventoId en Transferencia:** Cada transferencia nace con un UUID que se guarda en su fila.

Así, si el evento se publica dos veces, lleva el mismo id y después se puede detectar el duplicado. Agregué un @PostLoad para que las filas viejas de la S08, que no tenían eventoId, reciban uno al leerse.

- **EventoTransferencia**: Es un record aparte de la entidad que representa el hecho "la transferencia se realizó".
- **ConfiguracionMensajeria:** Declara el exchange transferencias, de tipo topic y durable, y el convertidor para mandar los eventos como JSON.
- **application.yml:** Activé publisher-confirm-type: correlated, publisher-returns y mandatory, y puse un connection-timeout de 2 segundos para no esperar 60 si el broker no contesta.
- **PublicadorDeEventos:** Manda el mensaje como persistente (se guarda en disco) y con el eventoId como messageId. Después espera la confirmación del broker. Si llega un nack, si se acaba el tiempo o si el broker regresa el mensaje porque ninguna cola lo recibió, lanza una excepción.
- **TransferenciaService.** Primero guarda la transferencia como COMPLETADA en la base y después publica el evento. Lo hice en ese orden para que nadie avise de algo que la base todavía no dice que pasó. En compensar publica TransferenciaRevertida cuando la saga queda en COMPENSADA. El método avisar atrapa cualquier error al publicar: si falla, solo lo registra en el log y la transferencia no se revierte, porque el dinero ya se movió bien.

  En la prueba con notificación apagado, la transferencia respondió 201 Created con estado COMPLETADA y con su eventoId (primera captura). En el log aparece que el evento no se publicó porque ninguna cola lo recibió (NO_ROUTE), y en el panel el exchange transferencias aparece como topic y durable, pero sin ningún binding. Esto es lo esperado en este punto, porque la cola la declara notificación hasta la sección 4. Ese primer intento con el servicio de movimiento tardó y dio un 503 por arranque en frío; por eso la transferencia 1 quedó en ABONADA y repetí la prueba, que ya salió con 201.



¿Qué garantiza el *publisher confirm*?

El publisher confirm garantiza que el broker recibió el mensaje y lo aceptó antes de que transferencia lo dé por publicado. Si el broker lo rechaza, no lo guarda, o ninguna cola lo recibe, transferencia se entera y lo deja en el log. En la prueba pasó justo eso: el broker devolvió el mensaje con NO_ROUTE y la transferencia lo registró.

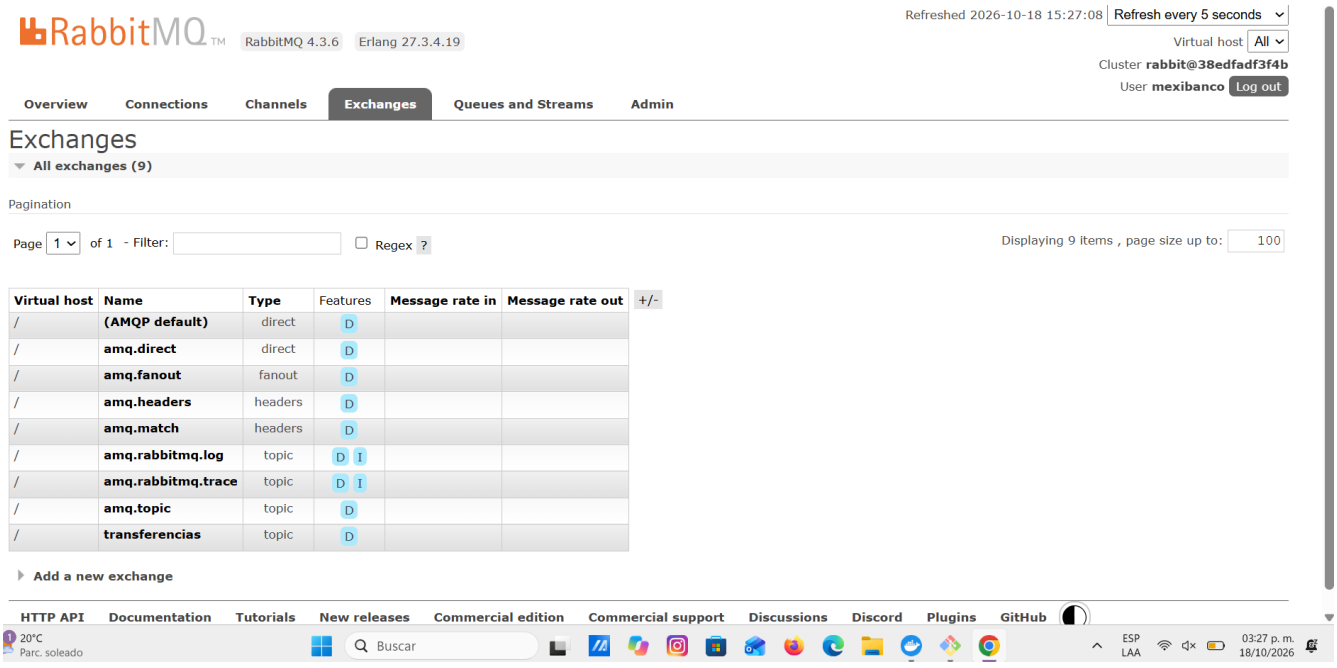
Lo que no garantiza: Primero que alguien ya lo haya leído. Con notificación apagado, el mensaje puede estar seguro en la cola y el aviso todavía no llega; eso es desacople en tiempo, no una falla. Segundo que el evento llegue a salir. Si el proceso de transferencia se muere justo entre guardar y avisar, la base dice COMPLETADA y el evento nunca se publica. El confirm no ayuda porque ni siquiera se llegó a mandar el mensaje.

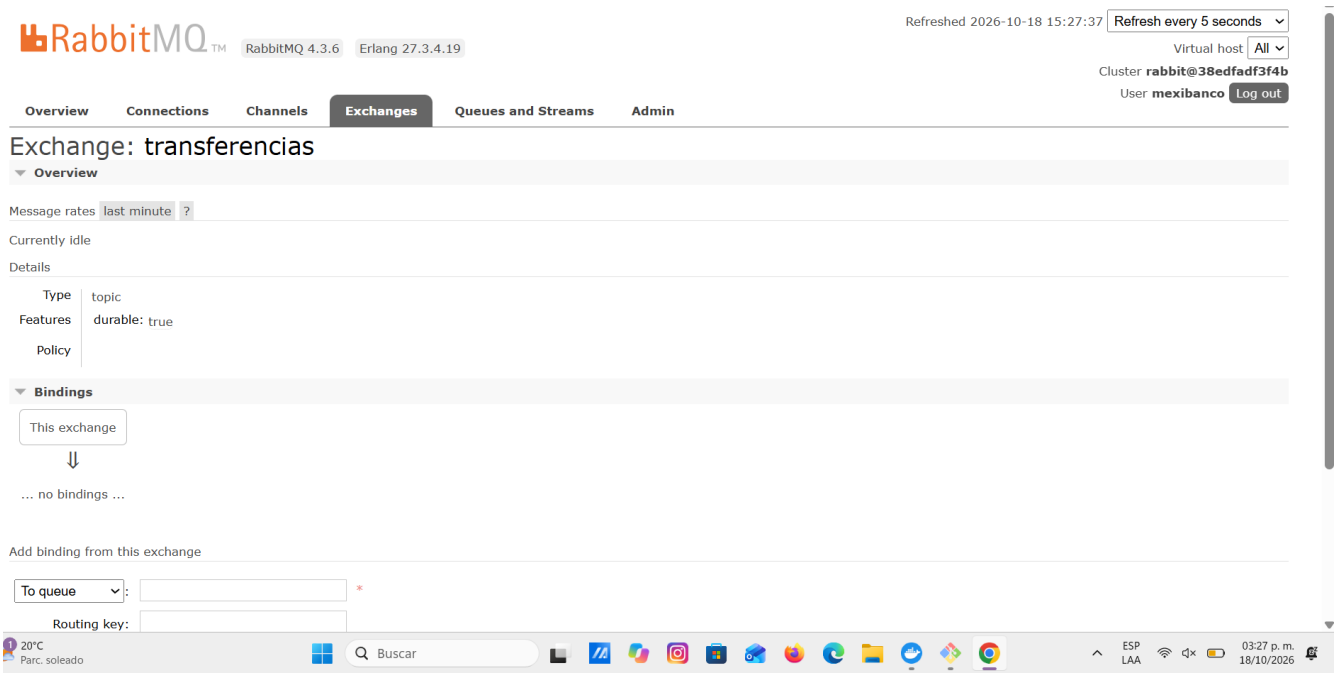
```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000002
{"id":1,"clabe":"002180000000000001","titular":"Ana","saldo":1000.00}{"id":2,"clabe":"002180000000000002","titular":"Beto","saldo":1000.00}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped
```

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000002
[{"id":1,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-18T21:23:48.107535Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-18T21:23:50.569315Z","eventoId":"c0455655-47fb-47a4-b823-9f265d98436e"}]{"id":1,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}{"id":2,"clabe":"002180000000000002","titular":"Beto","saldo":1050.00}
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 18 Oct 2026 21:30:23 GMT

{"id":2,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":"2026-10-18T21:30:23.449747331Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompensacion":0,"actualizada":"2026-10-18T21:30:23.651578173Z","eventoId":"8e04d37f-ac58-4226-83dd-2378d55f96ce"}
```

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-18T21:30:23.741Z ERROR 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.TransferenciaService   : Transferencia 2: quedo terminada pero el evento TransferenciaRealizada 8e04d37f-ac58-4226-83dd-2378d55f96ce NO se publico: Ninguna cola recibio el evento 8e04d37f-ac58-4226-83dd-2378d55f96ce con la routing key transferencia.realizada (NO_ROUTE)
```





3. Desacople en tiempo

Las tres transferencias con notificación apagado

Apagué notificación y mandé tres transferencias de 21, 22 y 23. Las tres respondieron 201. Antes, en la Parte 0, con el mismo servicio apagado la transferencia fallaba. Ahora la saga ya no espera a notificación.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ for m in 21 22 23; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"0021800000000000001\",\"claveDestino\":\"0021800000000000002\",\"monto\":$m}"
done
201
201
201
```

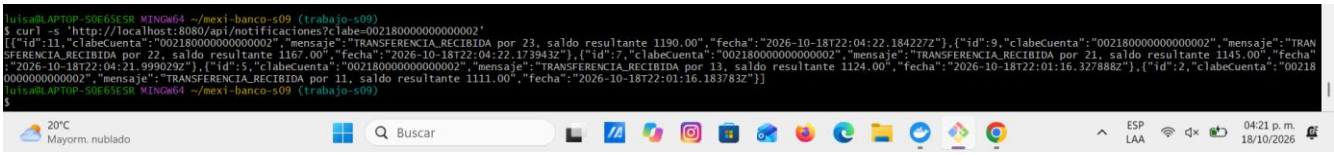
La cola con 3 Ready

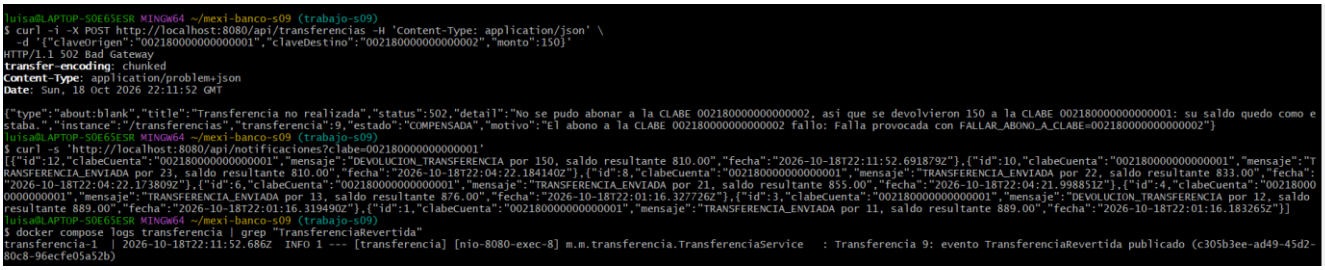
Con notificacion apagado, rabbitmqctl list_queues mostró notificacion.transferencias con 3 mensajes Ready y 0 Unacked. Los eventos se quedaron esperando en la cola durable.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
docker compose start notificacion
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
[+] start 4/4
 ✔ Container mexi-banco-v06-discovery-1    Healthy
 ✔ Container mexi-banco-v06-rabbitmq-1     Healthy
 ✔ Container mexi-banco-v06-db-1           Healthy
 ✔ Container mexi-banco-v06-notificacion-1 Started
```

La bandeja al volver

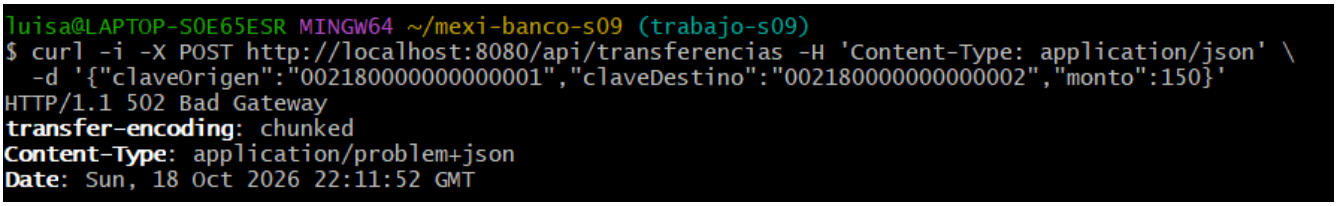
Al encender notificacion, el consumidor leyó los tres mensajes y la cola bajó a 0. En la bandeja de Beto aparecen los avisos TRANSFERENCIA_RECIBIDA por 21, 22 y 23, y en la de Ana, los TRANSFERENCIA_ENVIADA correspondientes. Esto es el desacople en tiempo: productor y consumidor ya no tienen que estr vivos al mismo tiempo.

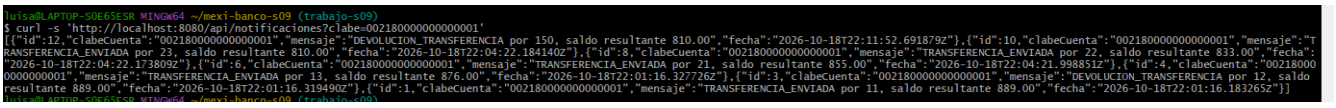


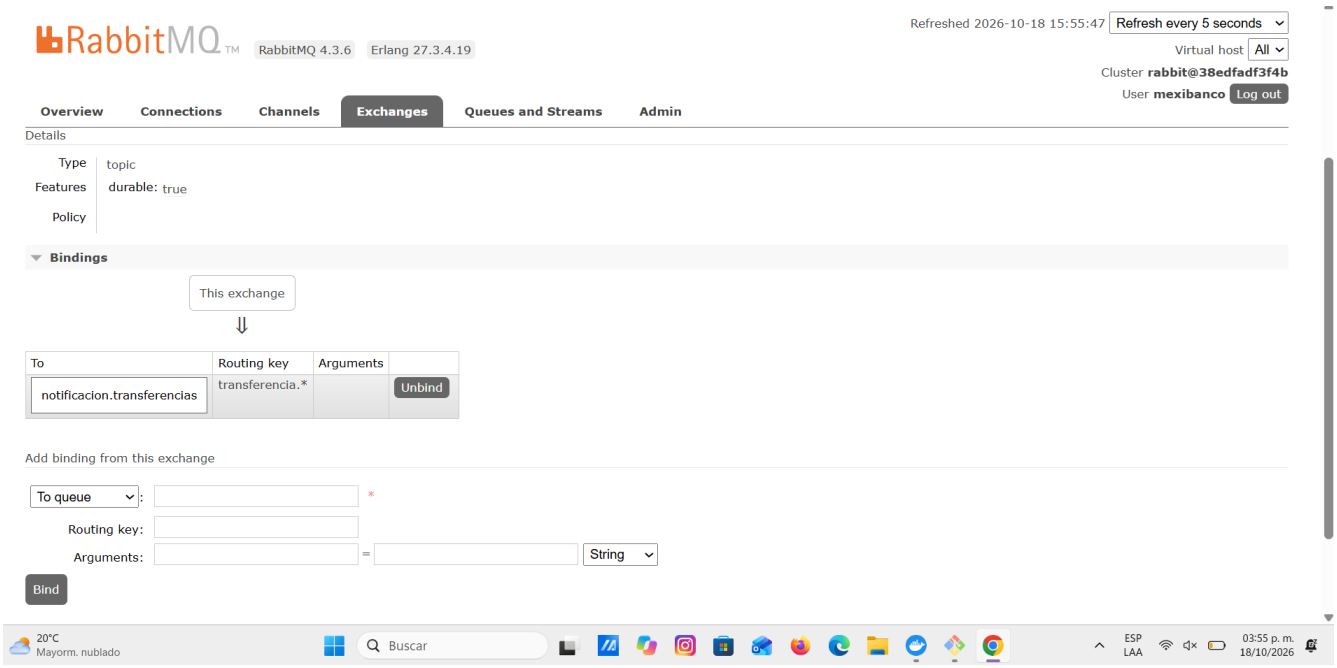


Aviso de TransferenciaRevertida.

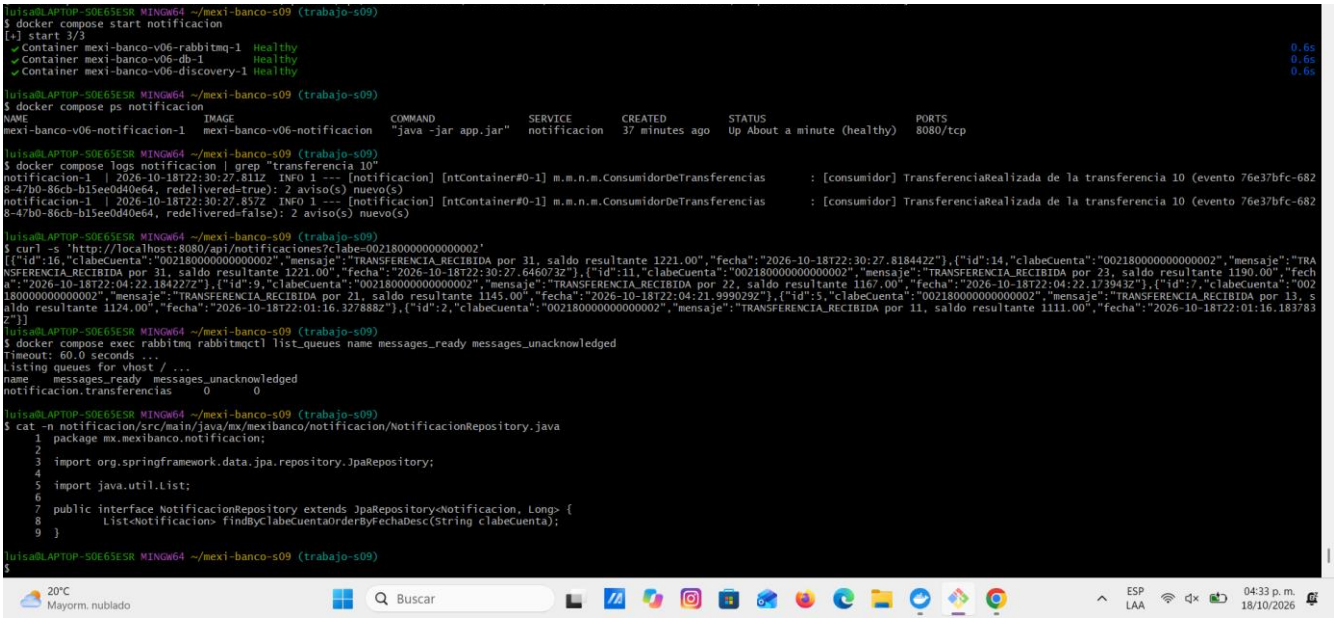
Para probar la revertida arranqué transferencia con FALLAR_ABONO_A_CLABE=002180000000000002, de modo que el abono a Beto fallara a propósito. Mandé una transferencia de 150 y la respuesta fue 502 con estado COMPENSADA: la saga devolvió el cargo y el saldo de Ana quedó como estaba. En el log de transferencia aparece evento TransferenciaRevertida publicado, y en la bandeja de Ana llegó un aviso nuevo, DEVOLUCION_TRANSFERENCIA por 150. Antes de la S09 nadie le avisaba al cliente cuando su transferencia se revertía.







4. El duplicado



Los avisos repetidos

Para provocar el duplicado apagué notificación y mandé una transferencia de 31 (transferencia 10). Desde el panel de RabbitMQ espié el mensaje de la cola con Nack message requeue true, copié su payload y lo publiqué otra vez en el exchange transferencias con la routing key transferencia.realizada, sin cambiar nada (mismo eventoId).

Al encender notificación, el log mostró dos entregas de la transferencia 10 con el mismo evento

76e37bfc-...: una con redelivered=true y otra con redelivered=false, cada una con “2 aviso(s) nuevo(s)”. En la bandeja de Beto quedaron dos avisos idénticos de TRANSFERENCIA_RECIBIDA por 31 (ids 14 y 16). El consumidor no recordaba qué eventos ya había procesado y por eso los guardó dos veces.

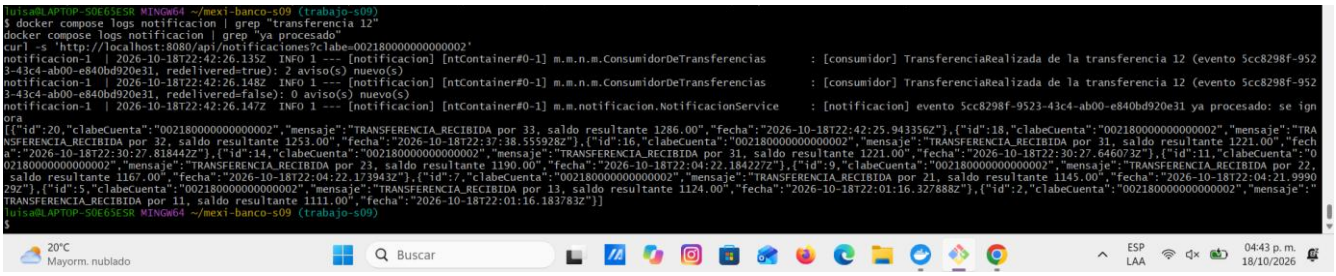
Mi arreglo

Hice el consumidor idempotente usando el eventoId: Creé la entidad EventoProcesado, con una tabla evento_procesado cuya llave primaria es el eventoId. También creé EventoProcesadoRepository con el método marcarSiEsNuevo, que ejecuta INSERT ... ON CONFLICT (evento_id) DO NOTHING y regresa cuántas filas insertó (1 si el evento era nuevo, 0 si ya existía). Al inicio de avisarDeTransferencia agregué una guarda: si marcarSiEsNuevo regresa 0, el método devuelve una lista vacía sin guardar nada y el consumidor hace el ack igual.

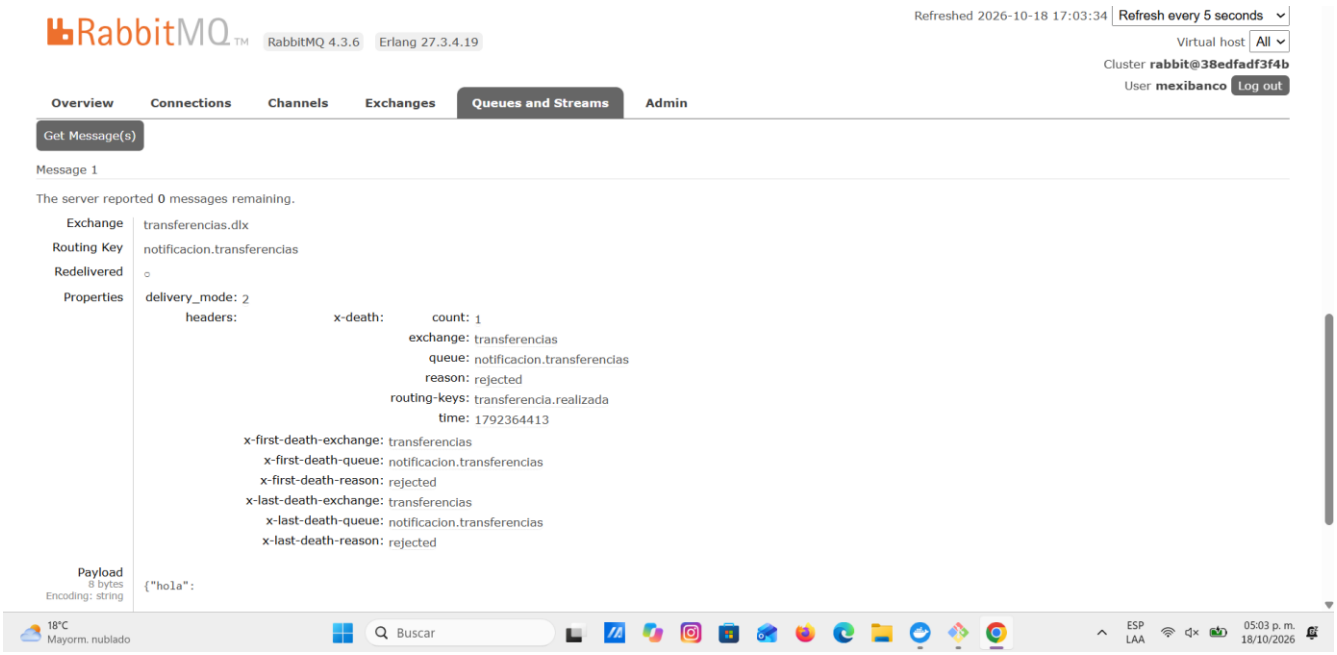
La marca y los avisos están en la misma transacción. Si algo falla después de marcar el evento, el rollback se lleva la marca y los avisos juntos, y la siguiente entrega empieza de cero. Usé un solo INSERT ... ON CONFLICT en lugar de consultar y luego insertar, porque entre esas dos operaciones otra entrega del mismo mensaje podría colarse.

Experimento repetido sin duplicado.

Repetí el experimento con otro monto (33, transferencia 12). El log mostró las dos entregas del evento 5cc8298f: la primera con 2 avisos nuevos y la segunda con 0 avisos nuevos, además de la línea evento ya procesado: se ignora. En la bandeja de Beto aparece un solo TRANSFERENCIA_RECIBIDA por 33 (id 20). El duplicado se detectó y se descartó.



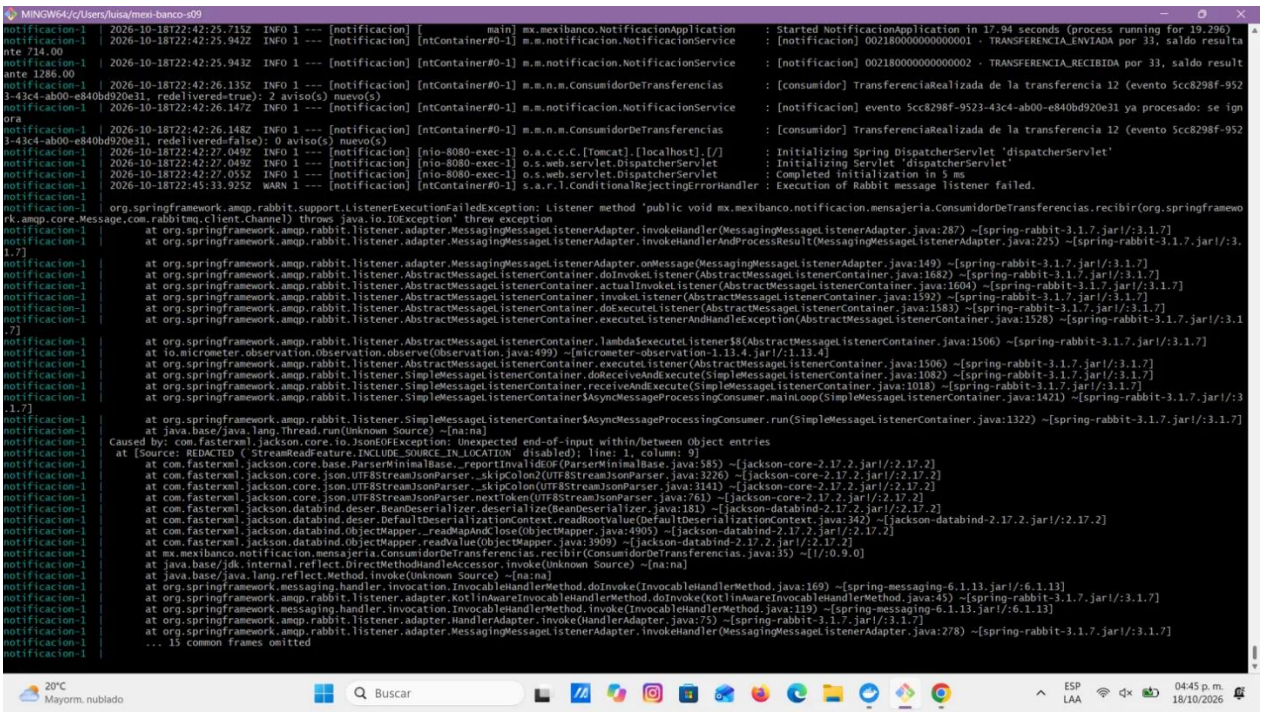
5. La DLQ

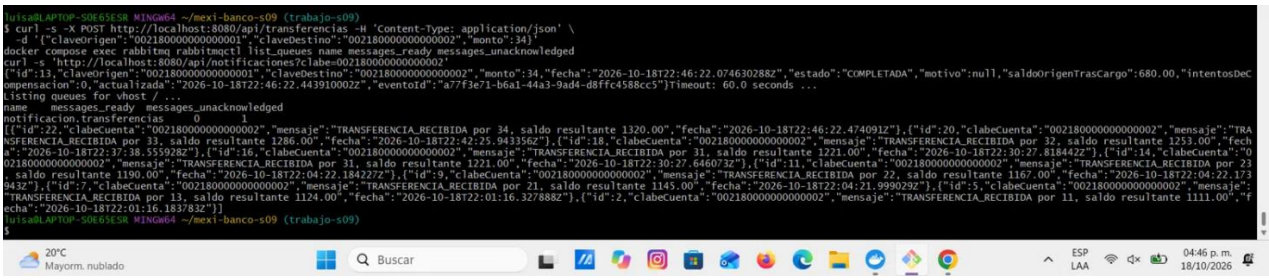


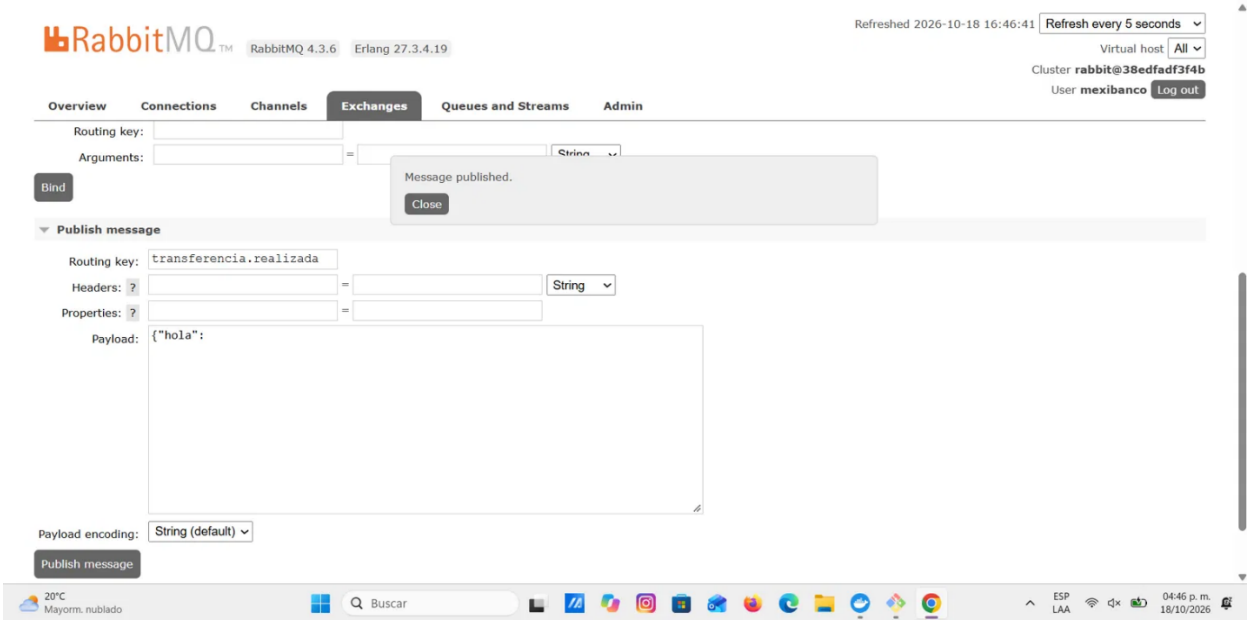
El mensaje malo Antes

Publiqué desde el panel un mensaje inválido, {"hola":, con la routing key transferencia.realizada. El consumidor lanzó una excepción de Jackson (JsonEOFException: Unexpected end-of-input) al leerlo, y el mensaje se quedó en Unacked: sin ack ni nack, el broker lo seguía considerando en manos del consumidor (list_queues mostró 0 Ready y 1 Unacked).

No rebotaba en bucle ni tiraba el servicio. Una transferencia normal que mandé después sí generó su aviso, pero solo porque el prefetch es 5. Cada mensaje malo ocupa uno de esos cinco lugares; con cinco así, la cola dejaría de avanzar.


MINGW64:/c/Users/luisa/mexi-banco-s09
notificacion-1 | 2026-10-18T22:42:25.715Z INFO 1 --- [notificacion] [main] mx.mexibanco.NotificacionApplication : Started NotificacionApplication in 17.94 seconds (process running for 19.296)
notificacion-1 | 2026-10-18T22:42:25.942Z INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService : [notificacion] 002180000000000001 · TRANSFERENCIA_ENVIADA por 33, saldo resulta
nte 714.00
notificacion-1 | 2026-10-18T22:42:25.943Z INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService : [notificacion] 002180000000000002 · TRANSFERENCIA_RECIBIDA por 33, saldo result
ante 1286.00
notificacion-1 | 2026-10-18T22:42:26.135Z INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias : [consumidor] TransferenciaRealizada de la transferencia 12 (evento 5cc8298f-952
3-43c4-ab00-e840bd920e31, redelivered=true): 2 aviso(s) nuevo(s)
notificacion-1 | 2026-10-18T22:42:26.147Z INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService : [notificacion] evento 5cc8298f-9523-43c4-ab00-e840bd920e31 ya procesado: se ign
ora
notificacion-1 | 2026-10-18T22:42:26.148Z INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias : [consumidor] TransferenciaRealizada de la transferencia 12 (evento 5cc8298f-952
3-43c4-ab00-e840bd920e31, redelivered=false): 0 aviso(s) nuevo(s)
notificacion-1 | 2026-10-18T22:42:27.049Z INFO 1 --- [notificacion] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
notificacion-1 | 2026-10-18T22:42:27.049Z INFO 1 --- [notificacion] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
notificacion-1 | 2026-10-18T22:42:27.055Z INFO 1 --- [notificacion] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 5 ms
notificacion-1 | 2026-10-18T22:45:33.925Z WARN 1 --- [notificacion] [ntContainer#0-1] s.a.r.l.ConditionalRejectingErrorHandler : Execution of Rabbit message listener failed.
notificacion-1 |
notificacion-1 | org.springframework.amqp.rabbit.support.ListenerExecutionFailedException: Listener method 'public void mx.mexibanco.notificacion.mensajeria.ConsumidorDeTransferencias.recibir(org.springframewo
rk.amqp.core.Message,com.rabbitmq.client.Channel) throws java.io.IOException' threw exception
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(MessagingMessageListenerAdapter.java:287) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandlerAndProcessResult(MessagingMessageListenerAdapter.java:225) ~[spring-rabbit-3.1.7.jar!/:3.
1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.onMessage(MessagingMessageListenerAdapter.java:149) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doInvokeListener(AbstractMessageListenerContainer.java:1682) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.actualInvokeListener(AbstractMessageListenerContainer.java:1604) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.invokeListener(AbstractMessageListenerContainer.java:1592) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doExecuteListener(AbstractMessageListenerContainer.java:1583) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListenerAndHandleException(AbstractMessageListenerContainer.java:1528) ~[spring-rabbit-3.1.7.jar!/:3.1
.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.lambda$executeListener$8(AbstractMessageListenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at io.micrometer.observation.Observation.observe(Observation.java:499) ~[micrometer-observation-1.13.4.jar!/:1.13.4]
notificacion-1 | at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListener(AbstractMessageListenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.doReceiveAndExecute(SimpleMessageListenerContainer.java:1082) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.receiveAndExecute(SimpleMessageListenerContainer.java:1018) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer$AsyncMessageProcessingConsumer.mainLoop(SimpleMessageListenerContainer.java:1421) ~[spring-rabbit-3.1.7.jar!/:3
.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer$AsyncMessageProcessingConsumer.run(SimpleMessageListenerContainer.java:1322) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at java.base/java.lang.Thread.run(Unknown Source) ~[na:na]
notificacion-1 | Caused by: com.fasterxml.jackson.core.io.JsonEOFException: Unexpected end-of-input within/between Object entries
notificacion-1 | at [Source: REDACTED (`StreamReadFeature.INCLUDE_SOURCE_IN_LOCATION` disabled); line: 1, column: 9]
notificacion-1 | at com.fasterxml.jackson.core.base.ParserMinimalBase._reportInvalidEOF(ParserMinimalBase.java:585) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon2(UTF8StreamJsonParser.java:3226) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon(UTF8StreamJsonParser.java:3141) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.core.json.UTF8StreamJsonParser.nextToken(UTF8StreamJsonParser.java:761) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.databind.deser.BeanDeserializer.deserialize(BeanDeserializer.java:181) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.databind.deser.DefaultDeserializationContext.readRootValue(DefaultDeserializationContext.java:342) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.databind.ObjectMapper._readMapAndClose(ObjectMapper.java:4905) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1 | at com.fasterxml.jackson.databind.ObjectMapper.readValue(ObjectMapper.java:3909) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1 | at mx.mexibanco.notificacion.mensajeria.ConsumidorDeTransferencias.recibir(ConsumidorDeTransferencias.java:35) ~[!/:0.9.0]
notificacion-1 | at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source) ~[na:na]
notificacion-1 | at java.base/java.lang.reflect.Method.invoke(Unknown Source) ~[na:na]
notificacion-1 | at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:169) ~[spring-messaging-6.1.13.jar!/:6.1.13]
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.KotlinAwareInvocableHandlerMethod.doInvoke(KotlinAwareInvocableHandlerMethod.java:45) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.invoke(InvocableHandlerMethod.java:119) ~[spring-messaging-6.1.13.jar!/:6.1.13]
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.HandlerAdapter.invoke(HandlerAdapter.java:75) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(MessagingMessageListenerAdapter.java:278) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1 | ... 15 common frames omitted
notificacion-1 |
20°C
Mayorm. nublado
Buscar
ESP
LAA
04:45 p. m.
18/10/2026


luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
-d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":34}'
docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
{"id":13,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":34,"fecha":"2026-10-18T22:46:22.074630288Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":680.00,"intentosDeC
ompensacion":0,"actualizada":"2026-10-18T22:46:22.443910022","eventoId":"a77f3e71-b6a1-44a3-9ad4-d8ffc4588cc5"}Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name messages_ready messages_unacknowledged
notificacion.transferencias 0 1
[{"id":22,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 34, saldo resultante 1320.00","fecha":"2026-10-18T22:46:22.474091Z"},{"id":20,"clabeCuenta":"002180000000000002","mensaje":"TRA
NSFERENCIA_RECIBIDA por 33, saldo resultante 1286.00","fecha":"2026-10-18T22:42:25.943356Z"},{"id":18,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 32, saldo resultante 1253.00","fech
a":"2026-10-18T22:37:38.555928Z"},{"id":16,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 31, saldo resultante 1221.00","fecha":"2026-10-18T22:30:27.818442Z"},{"id":14,"clabeCuenta":"0
02180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 31, saldo resultante 1221.00","fecha":"2026-10-18T22:30:27.646073Z"},{"id":11,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 23
, saldo resultante 1190.00","fecha":"2026-10-18T22:04:22.184227Z"},{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 22, saldo resultante 1167.00","fecha":"2026-10-18T22:04:22.173
943Z"},{"id":7,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 21, saldo resultante 1145.00","fecha":"2026-10-18T22:04:21.999029Z"},{"id":5,"clabeCuenta":"002180000000000002","mensaje":
"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 1124.00","fecha":"2026-10-18T22:01:16.327888Z"},{"id":2,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 1111.00","f
echa":"2026-10-18T22:01:16.183783Z"}]
luisa@LAPTOP-S0E65ESR MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$
20°C
Mayorm. nublado
Buscar
ESP
LAA
04:46 p. m.
18/10/2026

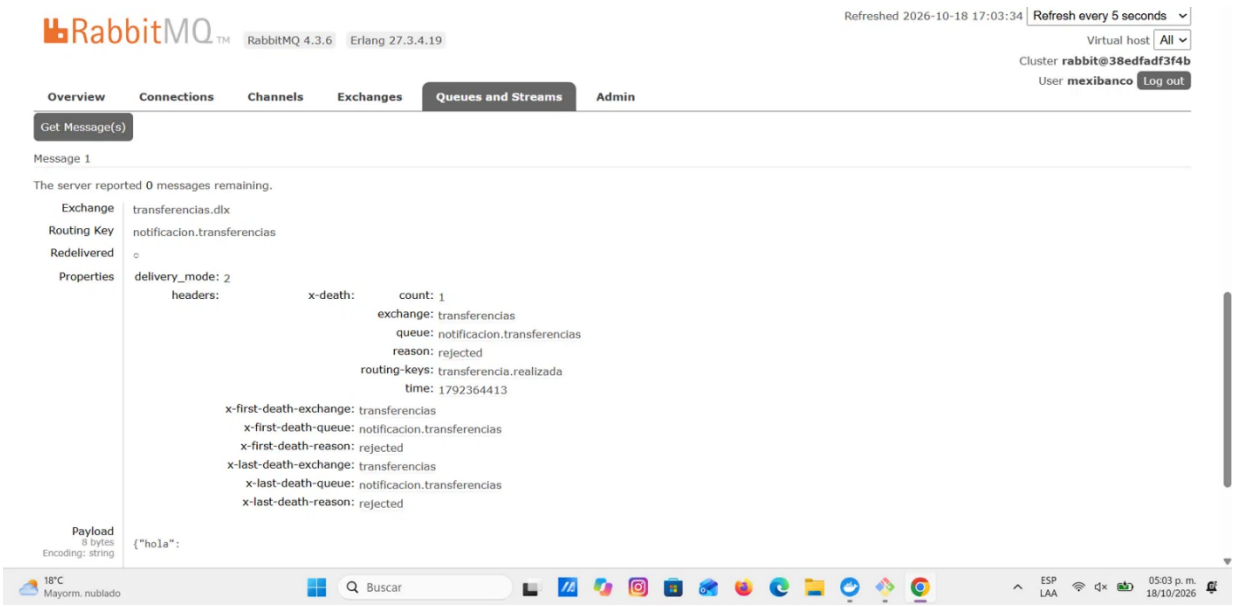

RabbitMQ™ RabbitMQ 4.3.6 Erlang 27.3.4.19
Refreshed 2026-10-18 16:46:41 Refresh every 5 seconds
Virtual host All
Cluster rabbit@38edfadf3f4b
User mexibanco Log out
Overview Connections Channels Exchanges Queues and Streams Admin
Routing key:
Arguments:
String
Bind
Message published.
Close
Publish message
Routing key: transferencia.realizada
Headers: ?
Properties: ?
String
Payload: {"hola":
Payload encoding: String (default)
Publish message
20°C
Mayorm. nublado
Buscar
ESP
LAA
04:46 p. m.
18/10/2026

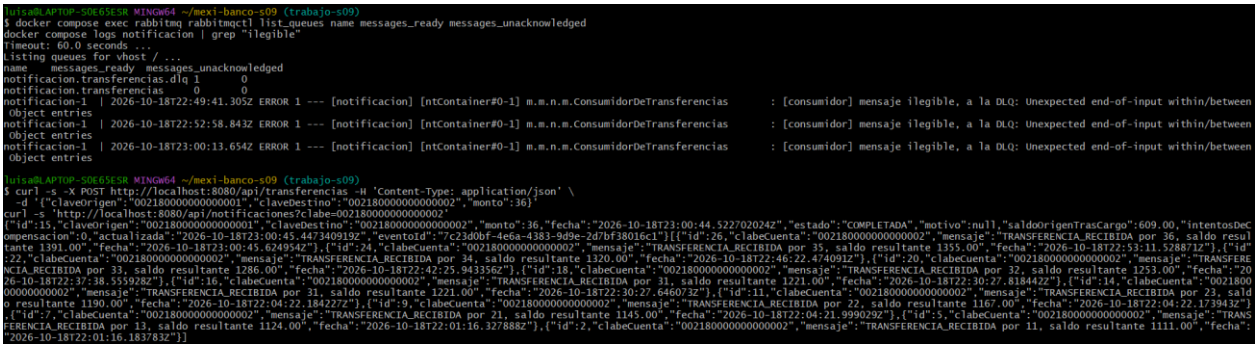
El mensaje malo Después

Configuré la dead letter queue. La cola notificacion.transferencias ahora lleva los argumentos x-dead-letter-exchange = transferencias.dlx y x-dead-letter-routing-key = notificacion.transferencias. Declaré el exchange transferencias.dlx (direct, durable) y la cola notificacion.transferencias.dlq, enlazada con la clave notificacion.transferencias. En el consumidor, si el JSON no se puede leer (JsonProcessingException), hago basicNack(etiqueta, false, false). El último false es requeue, así que no se reencola y es el broker quien lo mueve al DLX y de ahí a la DLQ.

Al arrancar me salió PRECONDITION_FAILED - inequivalent arg 'x-dead-letter-exchange', porque RabbitMQ no deja cambiar los argumentos de una cola que ya existe. Detuve notificacion, borré la cola vieja con rabbitmqctl delete_queue notificacion.transferencias y volví a levantar el servicio. Ahí se crearon las dos colas.

Después publiqué otra vez {"hola":. La cola principal quedó en 0 Ready y 0 Unacked, y la DLQ con 1 mensaje. El log dice “mensaje ilegible, a la DLQ”. Una transferencia normal (monto 36) generó su aviso (id 26), así que el mensaje malo ya no estorba.





El mensaje malo Con el panel

En el panel, con Get messages sobre notificacion.transferencias.dlq, se ve el mensaje con payload {"hola":, exchange transferencias.dlx, routing key notificacion.transferencias y la cabecera x-death con reason: rejected, queue: notificacion.transferencias y count: 1. Eso confirma que el broker lo apartó por haber sido rechazado.

Para pensar: un JSON bien formado con un tipo que no existe, o sin eventoId, también es veneno, porque reintentarlo nunca lo va a arreglar. En mi versión eso lanza una excepción en el servicio y no se manda a la DLQ; lo correcto sería atraparlo y hacer nack sin reencolar. En cambio, si lo que falla es la base de datos, el mensaje está bien y conviene reencolarlo, porque el reintento sí puede funcionar.

6. Preguntas de cierre

- **Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el *publisher confirm*?**

  Según cómo quedó mi código, si RabbitMQ está caído PublicadorDeEventos no logra conectarse o no recibe confirmación y lanza EventoNoPublicadoException. El método avisar la atrapa y solo escribe un error en el log. La transferencia ya estaba guardada como COMPLETADA, así que el cliente recibe su 201, pero el aviso nunca sale y se pierde, porque nada lo vuelve a intentar.

  El publisher confirm no lo arregla porque solo funciona cuando hay un broker que conteste: es el broker quien dice ya lo tengo. Si está caído, no hay quién confirme. Además, la base de datos y el broker son dos sistemas sin una transacción que los una, así que la base puede decir que se completó mientras el evento nunca salió. Eso se resolvería con un Transactional Outbox.

- **¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?**

  Ack después de guardar: si nnotificación muere entre guardar y el ack, el broker no recibe la

confirmación y vuelve a entregar el mensaje con redelivered=true. El aviso no se pierde, pero puede llegar dos veces. Obtengo entrega al menos una vez, y por eso hice el consumidor idempotente con eventoId.

Ack antes de guardar: si muere entre el ack y el guardado, el broker ya borró el mensaje y el aviso se pierde para siempre, sin que nadie se entere. Obtendría entrega a lo más una vez: nunca hay duplicados, pero puede haber pérdidas. Para un banco es más grave perder un aviso que recibirlo repetido, y el duplicado se puede controlar, por eso conviene el ack después.

- **Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.**
  Elección: cambiar la llamada HTTP síncrona por un evento asíncrono en RabbitMQ.

  Porque: así un aviso, que puede esperar, ya no tumba una transferencia que ya movió el dinero, y notificación puede estar caído sin afectar al cliente.

  Sacrifico: la certeza inmediata de que el aviso se entregó. Ahora hay consistencia eventual: el aviso puede llegar tarde, repetido (por eso agregué idempotencia) o perderse si RabbitMQ está caído y no hay Outbox. Además, aumentó la complejidad: más piezas que operar, como la cola, la DLQ y la tabla de eventos procesados.